

1. Objetivo de la práctica

En esta práctica el alumno seguirá trabajando indirectamente el control de versiones, además de:

- Uso básico de *javadoc*
- Refactorización usando un IDE
- Seguimiento de guías de estilo y eliminación de *bad smells*

Esta práctica será tenida en cuenta para el RA4.

2. Enunciado

Este programa resuelve el problema del **pentominó**. El código fuente está bastante bien, pero es necesario mejorarlo en algunos aspectos. En concreto, hay que mejorar ciertas partes internas del código y no está documentado adecuadamente. El código fuente inicial está en <https://codeberg.org/alvarogonzalezsotillo/pentominos>

2.1. Documentación (2 puntos)

Hay clases, atributos y métodos sin documentar. Se debe completar la documentación (en español o en inglés, indistintamente)

- Todas las clases tendrán `@author` (usa tu nombre y apellido). Existen desde la versión 1.0.
- Todas las clases tendrán alguna referencia `@see` a otra que se use o por la que sea usada.
- En la clase `Pentominos`
 - Todos los métodos tendrán `@param` para todos sus atributos. Todos los métodos con retorno tendrán `@return`
 - Cada método tendrá una referencia `@link` a algún método que sea llamado dentro de él. Si no llama a ningún método, tendrá una referencia a un método que lo llame.

2.2. Refactorización (4 puntos)

Realiza estos cambios con las herramientas de refactorización de IntelliJ:

- Las normas de codificación de nuestra metodología indican que los atributos de las clases comienzan con el prefijo `m_`. Todos los atributos de la clase `MosaicPanel` se modificarán.
- El `PentominosPanel.menuHandler` es demasiado largo. Cada acción se realizará en un método aparte, en vez de dentro del `if` encadenado.
- El método `PentominosPanel.doSaveImage` se dividirá en dos:
 - Una parte pedirá un fichero y comprobará que no existe
 - La otra parte utilizará ese fichero para grabar la imagen.
- Los atributos `rows` y `columns` de `MosaicPanel` no se accederán directamente. Se utilizarán métodos `set` y `get` para ello.
- Las clases deberán formar parte del paquete `edu.hws.godel`.

2.3. Clean code (3)

- Consigue que la clase `MakeIcons` no tenga ningún aviso de Sonarqube.
- Puedes ignorar el aviso `S106`

2.4. Javadoc (1 punto)

- Genera la documentación *javadoc* del proyecto
- Se dejará en el directorio `javadoc`

3. Normas de entrega

1. Se creará un repositorio en Codeberg, siendo inicialmente un *fork* de <https://codeberg.org/alvarogonzalezsotillo/pentominos>
2. La práctica se realizará en la rama `2026-refactorizacion-documentacion`
3. Se modificará el código sobre ese repositorio hasta ajustarse a lo pedido en la práctica.
4. Se creará un *tag* con la versión para entregar, de nombre `2026-entrega`
5. En el aula virtual, se subirá:
 - La URL del repositorio
 - El identificador del *commit* del *tag* de la entrega.
6. Si la práctica se realiza por parejas, todos los integrantes subirán el resultado al aula virtual, y en la historia del repositorio podrá verse que el reparto de trabajo ha sido equilibrado

4. Qué se valorará

- Que haya un *commit* por cada subapartado.
- Que el código final se ajuste a lo pedido
- Que el código funcione (clases `Main`: `Pentominos` y `MakeIcons`)